



Fundamentos de Análisis de Imágenes

Práctica Final

Grupo: TDAH

Autores:

Daniel Moraleda y Víctor Pastor

Ciencia de Datos e Inteligencia Artificial

Índice

1.	INTRODUCCIÓN A LA PRÁCTICA.....	3
2.	DECISIONES DE DISEÑO.	4
3.	OBJETIVO Y BASELINE.....	6
4.	DIFICULTADES ENCONTRADAS.....	8
5.	PRIMER MODELO.	9
6.	SEGUNDO MODELO.....	12
7.	ANONIMIZACIÓN EN VÍDEO Y TIEMPO REAL.....	16
8.	CONCLUSIONES.	17
9.	BIBLIOGRAFÍA.	18

1. Introducción a la práctica.

Vivimos en una época en la que las redes sociales, la privacidad y dispositivos electrónicos son una base fundamental del día a día de prácticamente cualquier persona.

Compartimos nuestros viajes, nuestras comidas, e incluso a personas cercanas que, sin saberlo y por cuenta ajena, ven como su rostro entra en el *dataset* de la privilegiada empresa a la que elijamos compartir arbitrariamente nuestros datos.

Nos movemos en un mundo en el que nos arriesgamos a que la gente reconozca donde estamos, que hacemos o nos localicen por fotos usadas por otros.

El anonimato se ha perdido completamente. ¿Qué hago si quiero seguir manteniendo mi privacidad, aunque aparezca en una imagen?

El objetivo de esta práctica será crear un algoritmo que un usuario pueda usar para desenfocar un rostro en una imagen.

Además, lo que estamos buscando no radica únicamente en nuestro fin, sino que también depende de nuestro medios. En unos años donde el *Deep Learning*, la *Inteligencia Artificial* y el *Machine Learning* han colapsado todos los recursos y formas de predicción y detección, queremos alcanzar nuestro objetivo de manera puramente matemática.

Usaremos la librería de procesamiento de imágenes clásica por antonomasia, *OpenCV* y trataremos de, usando solo sus métodos y técnicas matemáticas, conseguir el mejor resultado posible.

2. Decisiones de diseño.

Aunque este apartado podría parecer baladí, en realidad es una reflexión clave para el correcto desarrollo de la práctica y extraer el máximo potencial de nuestra idea. Queremos crear una herramienta centrada en la utilidad real.

1. ¿Qué debe hacer el algoritmo?

Queremos que la herramienta reciba una imagen en la que aparezcan una o varias personas y que esta seleccione las caras de las personas que se encuentren. Tras esto, se le preguntará al usuario cuales son las caras que se quieren anonimizar y se desfocarán.

2. ¿Qué estamos buscando?

Tras la decisión del funcionamiento de la herramienta, llega la segunda gran pregunta que debemos hacernos, cómo valoramos la eficacia de nuestro algoritmo y en que nos enfocamos para su desarrollo.

A diferencia de otros problemas de aprendizaje automático, aquí no podemos confiar en la exactitud (*accuracy*). Debido a la naturaleza de las imágenes, sufrimos un desequilibrio de clases: los píxeles correspondientes a rostros suelen representar apenas entre el 0,5% y el 5% del total de la imagen (salvo en primeros planos extremos, donde podrían llegar al 30%). Por tanto, un algoritmo que simplemente predijera 'fondo' para todos los píxeles tendría una exactitud del 95-99%, siendo sin embargo totalmente inútil.

Estos datos no son arbitrarios y han sido extraídos mediante muchas pruebas realizadas. Algunos ejemplos para poner en perspectiva (fotos extraídas de Pinterest):



La cara supone el 2.7% de la imagen



Las caras suponen el 2.3% de la imagen

La *precisión* y el *recall* son mucho más útiles para poder hacer una valoración de cómo funciona nuestro algoritmo.

Vamos a poner dos ejemplos simples para entender estas medidas:

- En medicina (prioridad *recall*): Es preferible un falso positivo (diagnosticar cáncer a alguien sano, que luego se descarta con pruebas) a un falso negativo (no detectar una enfermedad real).
- En justicia (prioridad *precisión*): Bajo la presunción de inocencia, se prefiere absolver a un culpable (falso negativo) antes que condenar a un inocente (falso positivo).

En nuestro caso, la inclusión de un humano final que decida cambia las reglas del juego. Si la herramienta fuera totalmente autónoma, necesitaríamos una alta precisión para evitar desenfocar objetos incorrectos. Sin embargo, dado que el usuario selecciona qué candidatos anonimizar, nuestro objetivo es maximizar el *recall* (aun así, queremos mantener también una buena precisión).

Preferimos que el sistema detecte erróneamente una rodilla o una lámpara como un rostro (un error que el usuario descartará), antes que dejar pasar una cara real sin detectar. Buscamos un *recall* cercano al 100% para garantizar la privacidad, delegando la precisión final en la selección del usuario.

3. Objetivo y baseline.

Como hemos explicado en el diseño, queremos priorizar la identificación del mayor número de caras posibles en la imagen usada, de tal forma que el usuario pueda elegir cuales desenfocar.

Más allá de la funcionalidad, este proyecto tiene un componente pedagógico que queremos poner por delante. Vamos a tratar de explotar al máximo los fundamentos de análisis de imagen clásico para encontrar el máximo potencial de los métodos estudiados en la asignatura. Somos plenamente conscientes de que, si añadiésemos componentes de librerías externas de *Deep learning* o modelos de selección pre-entrenados, los resultados mejorarían.

Por ello, el desarrollo se basará exclusivamente en algunas técnicas matemáticas y los algoritmos de procesamiento digital disponibles en la librería *OpenCV* como las máscaras de canales de color, las operaciones morfológicas o la geometría de contornos.

1. ¿Cuál será nuestro objetivo?

Vamos a tratar de acercarnos o mejorar lo máximo posible un algoritmo clásico del análisis de imagen: el *Haar Cascade*.

En primer lugar, vamos a entender cómo funciona este método estándar muy usado:

- El algoritmo busca patrones de contraste mediante características rectangulares (por ejemplo, la zona de los ojos suele ser más oscura que la zona de las mejillas, o el puente de la nariz más claro que los lados).
- Se dice que tiene una arquitectura en cascada porque tiene varias etapas de filtrado, lo que aumenta su velocidad. Si una región de la imagen no pasa la primera etapa (rasgos básicos), se descarta. Solo las regiones que pasan todos los filtros se consideran caras.

Sin embargo, al basarse en la morfología facial de frente, suele tener problemas con rotaciones de la cara o mala luz de la imagen. Por ello, queremos comprobar si un enfoque basado en una segmentación mayor por color y morfología puede obtener un *recall* competitivo o superior en situaciones donde la detección puramente basada en patrones de contraste (*Haar*) podría fallar.

2. Baseline:

Primero, estableceremos un modelo como baseline que intentaremos mejorar poco a poco añadiendo más complejidad al modelo. Nuestra idea inicial será la siguiente:

- En primer lugar, para preprocesar la imagen inicial, convertiremos la imagen de *BGR* a *HSV* para encontrar en que rango específico de los canales se encuentran los tonos de piel que estamos buscando.
- Como paso dos, crearemos un umbral para crear una máscara binaria con los tonos de piel seleccionados, de tal forma que lo identificado como piel quede blanco y todo lo demás, negro.
- Tras esto, para eliminar el ruido de objetos sueltos e imperfecciones, haremos una erosión y una dilatación (*MorphOpen*) y viceversa (*MorphClose*) y otra dilatación extra para rellenar los huecos de la cara no detectados como piel.
- En el cuarto paso, realizaremos una detección de bordes para reconocer la forma del rostro, buscando una forma no muy pequeña con una buena relación ancho – alto (*aspect ratio*) y una buena circularidad.
- Por último, realizaremos el desenfoque pegando un desenfoque gaussiano del rostro sobre la región en la que se encuentra.

4. Dificultades encontradas.

Como esta versión es preliminar, se han desenfocado todos los rostros detectados sin aplicar la selección del usuario. Nuestro objetivo actual es mejorar al máximo la detección. Tras este primer acercamiento al problema, nos hemos encontrado con varios problemas que van a ser nuestro centro de atención mejorar:

- **Falsos positivos:** Si hay elementos del mismo color que el rostro (como manos, brazos...) con forma parecida, seleccionar por la mayor área no nos selecciona el rostro. Deberíamos filtrar por otras cualidades matemáticas más específicas.
- **Falsos negativos:** este es el aspecto que debemos minimizar para conseguir el mejor *recall* posible. Actualmente, hay muchas caras que no están siendo reconocidas. Aunque empeoremos la métrica de precisión e introduzcamos falsos positivos, seremos menos agresivos con la selección para captar todas las caras.
- **Iluminación:** una saturación muy agresiva o una luz muy cercana que emblanquezca la cara hace que se salga del rango escogido y no se reconozca correctamente. Debemos encontrar la máscara de color que más se ajuste a lo que queremos.
- **Máscara de color y operaciones morfológicas:** podemos ver cómo se están uniendo elementos del fondo y de los rostros, lo que empeora las métricas y hace que muchas caras no se reconozcan. Trataremos de encontrar la mejor forma para separar los elementos de la imagen para poder hacer un análisis riguroso.

5. Primer Modelo.

Para este primer código, nos centraremos en el enfoque inicial que teníamos sobre el uso selectivo de la máscara y el uso de muchas propiedades matemáticas para ayudarnos a seleccionar las caras.

Consta de un modo de detección general y otro modo de aplicación del desenfoque selectivo. El primer modo sirve para ver el funcionamiento del algoritmo con muchos ejemplos a la vez, mientras que el segundo modo sirve para que el usuario pueda jugar con una imagen de su elección para que se le aplique el desenfoque de la cara a las personas que considere.

La estrategia seguida por el código es la siguiente:

1. Preprocesamiento de Imagen

En primer lugar, para preprocesar la imagen de entrada inicial, aplicamos un suavizado gaussiano y convertimos la imagen a los espacios de color *YCrCb* y *HSV* para separar la información de luminosidad del color de la piel.

El espacio *HSV* permite separar la imagen en colores, permitiendo que los colores de cierto todo estén en cierta franja de la capa *H*, la saturación en la capa *S* y la luminosidad en la capa *V*. Además, el espacio *YCrCb* muestra el factor de luminosidad de la imagen, permitiendo agrupar aquellos contornos con alto brillo (como la piel de las personas).

2. Creación de la máscara binaria

Creamos una máscara binaria robusta mediante rangos específicos en las capas *BGR*, *YCrCb* y *HSV*, de tal forma que un *píxel* solo es blanco está en la franja seleccionada en cada capa. Se deberá hacer una máscara por cada capa y juntarlas siguiendo la regla lógica *AND*.

3. Limpieza de la máscara

Calculamos un tamaño de *kernel* dinámico (basado en la dimensión de la imagen) para realizar operaciones morfológicas de *OPENING* (dilatación + erosión) y *CLOSING* (erosión + dilatación) asegurando que la limpieza funcione igual en imágenes grandes o pequeñas.

4. Creación y selección de contornos

Extraemos los contornos y calculamos un *score* (puntuación) para cada candidato basado en su relación de aspecto, solidez y circularidad, filtrando los que tienen baja puntuación y seleccionando solo aquellos cuyo tamaño es coherente con el promedio de las mejores caras detectadas.

Para ello seleccionamos distintas reglas para asignarle el *score* a cada contorno, usando su posición (x, y) y su ancho / alto (w, h) , con las que podemos hallar las métricas geométricas, como la relación entre el ancho y el alto (alto mejor que ancho), el perímetro, lo circular o lo sólido (huecos) que es.

- Si el área del contorno es muy pequeña (menos de un 0.2% del total), lo descartamos.
- Seguiremos una puntuación ponderada basada en las métricas, dando un 40% de peso a la relación ancho/alto, un 30% a lo sólido que es y un 30% a que tan circular es:

$$\text{score} = 0.4 * (1 - \text{abs}(\text{aspect} - 0.75)) + 0.3 * \text{solidity} + 0.3 * \text{circ}$$

Una vez tengamos el sistema de puntuación para ver qué tan bueno es un contorno, los ordenamos para encontrar los mejores candidatos. A partir de los 3 mejores candidatos, obtenemos el área de referencia para los rostros, que nos indica que tamaño deben tener, pudiendo eliminar los que sean muy pequeños.

A cada candidato adecuado se le hará un *padding*, para aumentar un 20% el tamaño del rectángulo y así garantizar cubrir la totalidad del rostro.

5. Aplicación de Desenfoque

Cuando tenemos los candidatos finales, solo quedaría aplicar el desenfoque al sector de la imagen que corresponde con dicho candidato.

Para ello usaremos una máscara elíptica para captar la forma del rostro, permitiendo que el desenfoque quede integrado en la imagen de forma natural.

Cuando se active la opción *apply_blur* en la función, se le pedirá al usuario que seleccione los candidatos que se quiere desenfocar

Cuando ya tengamos la máscara elíptica aplicaremos un desenfoque gaussiano bastante elevado, impidiendo que se pueda reconocer el rostro de la persona.

6. Ejemplos de funcionamiento

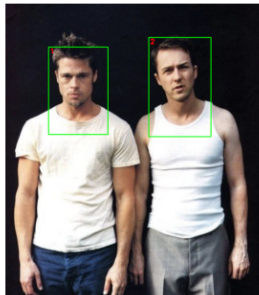
Original



Máscara de piel



Detecciones



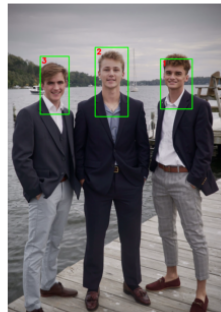
Original



Máscara de piel



Detecciones



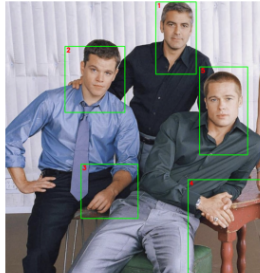
Original



Máscara de piel



Detecciones



6. Segundo Modelo.

Como segundo modelo, tenemos una aproximación algo distinta a la anterior. Al igual que en el anterior modelo, nos basaremos en la creación de una clase donde aplicaremos una máscara y unas reglas geométricas, pero desde un punto de vista distinto.

La estrategia seguida es la siguiente:

1. Preprocesamiento de Imagen (*Corrección Gamma y LUT*).

En primer lugar, aplicamos una corrección gamma sobre la imagen inicial para normalizar la iluminación. Más tarde explicaremos que este algoritmo sirve tanto para imágenes como vídeos, por lo que la entrada es o una imagen o un *frame*. Las cámaras captan la luz de forma lineal, pero el ojo humano no; además, en situaciones de contraluz o sombra, la piel pierde sus características de color.

Para aplicar esta corrección, utilizamos una *LUT* (*Look up table*). Calcular la potencia matemática pixel a pixel para una imagen *HD* (2 millones de píxeles) sería demasiado costoso, por lo que pre-calculamos con la *LUT* los 256 valores posibles en un array y simplemente mapeamos los valores.

Finalmente, aplicamos un suavizado gaussiano. Esto es muy importante para que la detección de color no sea tan sensible al ruido y la máscara final sea lo menos sucia posible.

2. Segmentación en Espacios de Color (*YCrCb y HSV*).

Convertimos la imagen original (*BGR*) a dos espacios de color distintos para que la máscara sea lo más robusta posible y no dependa solo de un punto de vista.

El espacio *YCrCb* separa el factor lumínico del color. Esto puede ser muy importante para detectar piel humana porque el tono tiende a agruparse en una región muy específica del plano *Cr-Cb*, independientemente de si la persona es de tez clara u oscura o de la intensidad de la luz (*Y*).

Por otra parte, *HSV* complementa a lo anterior, permitiéndonos filtrar por matriz y saturación. Nos ayuda a descartar objetos que tienen el color correcto pero que son demasiado pálidos y viceversa.

Generamos una máscara binaria para cada espacio y aplicamos una intersección lógica *AND*. Un píxel solo se vuelve blanco si ambos algoritmos coinciden, lo que es muy importante para reducir los falsos positivos, aumentando el ratio de precisión.

3. Refinamiento Morfológico (Operaciones y Kernel).

La máscara suele tener imperfecciones como huecos en la cara (ojos y boca) y mucho ruido en el exterior. Por ello, es necesario hacer algunas operaciones que mejoren nuestra máscara en pro de facilitar la selección de rostros.

En primer lugar, realizaremos un *CLOSING* (dilatación + erosión) para fusionar regiones cercanas y huecos de la cara para que los objetos sean sólidos.

En segundo lugar, aplicaremos un *OPENING* (erosión + dilatación) que nos ayudará a eliminar el ruido del fondo sin reducir el tamaño de las caras.

En este caso, dada la restrictiva máscara que hemos usado muchos huecos quedan por rellenar, por lo que hemos hecho tres iteraciones de la primera operación y una de la segunda, usando en ambas un *kernel* elíptico. Dado que los rostros humanos tienen curvas, un *kernel* cuadrado introduciría mucho ruido y bordes rectos que no corresponden a la realidad y que introducirían más ruido.

4. Separación de núcleos (Transformada de la Distancia).

Un problema que hemos tenido con el primer método y con el baseline es que, si dos personas están juntas, o el fondo es de un color reconocido por la máscara, esta se fusiona y produce blobs gigantes que el sistema reconoce como una única cara deforme.

Para evitar esto en la medida de lo posible, hemos usado la *Transformada de la Distancia*. Esta calcula las distancias de cada píxel blanco hasta el píxel negro más cercano. Los píxeles en el centro de la cara tienden a tener valores altos y los de los bordes bajos (intuitivamente, los bordes se llaman bordes porque están al exterior de una forma, ósea cerca de la parte negra descartada anteriormente, luego la distancia a un píxel negro es menor). Normalizamos las distancias para tener poder trabajar con medidas absolutas y los quedamos con el 20% de los valores máximos (estamos quedándonos con los núcleos de los rostros).

5. Selección de candidatos (Puntuación Geométrica).

Para determinar si un contorno es una cara, hemos usado algunas heurísticas geométricas.

La primera de ellas es el *aspect ratio* (ancho / alto de la cara). Si este *ratio* es muy alto (forma horizontal) o muy bajo (forma vertical), penalizamos la puntuación. En este caso hemos tendido a buscar ratios entre 0.55 y 0.95, aunque esto son parámetros que han sido ajustados a mano. Esta métrica es muy útil para evitar reconocer como caras brazos, piernas o palos.

En segundo lugar, hemos buscado contornos que sean lo más sólidos posibles, es decir, que cubran lo máximo posible del contorno que dibujan. Una mano por ejemplo tendría huecos y tendría una mala solidez. Una estructura con pilares y huecos intermedios también.

También penalizamos las regiones que tocan el borde inferior, ya que suelen ser piernas o suelo (es muy improbable que aparezca una cara entrecortada por abajo).

Cabe destacar que hemos descartado ruido muy pequeño (área del 0.015% del total) y también todos los blobs gigantes que se hayan podido producir (área del 15% del total).

7. Generación de *ROI* y anonimización.

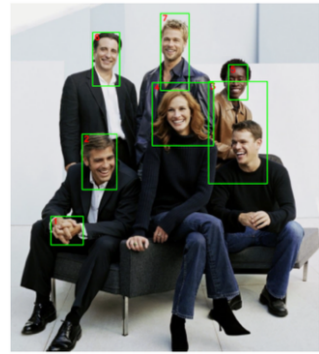
Tras las métricas anteriores, cada candidato ha conseguido un score. Si este ha sido mayor o igual que 5, se aceptará como una cara. Si fuésemos más restrictivos podríamos tener menos falsos negativos pero nuestra prioridad es el *recall*.

La máscara solo ha detectado la piel expuesta así que aumentamos el *ROI* en un 20% para tener un rectángulo que cubre la identidad de la persona al completo.

Por último, si el parámetro *apply_blur = True*, se aplica el desenfoque. Como hemos explicado anteriormente, no aplicamos un desenfoque cuadrado simple por estética. Generamos una máscara elíptica con un degradado y la mezclamos con la imagen original. Con esto hemos tratado de provocar un *fade* suave y estético en lugar de un corte limpio.

8. Ejemplos de funcionamiento.





7. Anonimización en vídeo y tiempo real.

Nuestra función de desenfoque consigue desenfocar rostros en imágenes de una forma bastante decente, lo que da paso a pensar que se podría hacer con videos.

Como sabemos, los videos son secuencias de *frames* (imágenes) mostradas cada poco tiempo. Nuestra estrategia para poder conseguir un video en el que se desenfoque a la persona es dividir el video que queremos transformar en una secuencia de sus *frames*. Una vez dividido en imágenes, aplicamos nuestra función a cada una de estas y las volvemos a juntar, formando el video transformado.

Para poder recrear el video sin fallos, debemos guardar los *FPS* del video original y calcular el tiempo (*ms*) de espera entre imágenes ($1000/n_frames$).

Además, esta misma lógica se puede aplicar a la cámara de nuestro portátil, permitiendo desenfocar en tiempo real capturando imágenes en tiempo real.

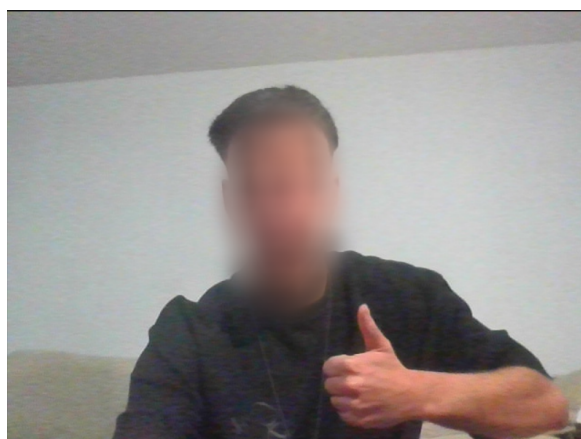
Resultados:

Algo para tener en cuenta es que las imágenes que procesamos se desenfocan de forma independiente, por lo que dependemos de que no haya movimientos bruscos entre *frames*. Por lo general, si no usamos videos con pocos *FPS* o evitamos movimientos rápidos los resultados son muy buenos. Además, si se captura la cámara o se usan video en los que haya buena iluminación, el resultado es mejor.

Ejemplos de video:



Ejemplo de capturas en tiempo real:



8. Conclusiones.

La resolución de la práctica que se había planteado se cumple con el primer método. La realidad es que el funcionamiento es muy bueno y hay un *recall* muy alto y unos falsos positivos aceptables, consiguiendo que el usuario pudiese seleccionar que rostros difuminar.

Sin embargo, hemos querido saber cuál es el límite de nuestras posibilidades con estas herramientas y este tiempo. Tras numerosas pruebas con este primer método, probando todo tipo de parámetros, operaciones morfológicas más agresivas o una búsqueda distinta de candidatos, los resultados no mejoraron.

Por ello, tratamos de buscar un enfoque distinto en el segundo método.

Esta nueva aproximación funciona incluso mejor, aunque hemos creado la máscara de forma muy distinta y hemos sido mucho menos agresivos con las restricciones. Esto demuestra que no hay una forma perfecta y exacta de abordar el problema, sino que se puede tratar de buscar una solución desde muchos puntos de vista distintos, siempre y cuando prevalezca el rigor lógico y matemático de lo que estamos haciendo.

En ambos casos, somos plenamente conscientes de los límites de nuestros modelos. En las pruebas hemos podido ver que, si hay muchos elementos en el entorno del mismo color que las características faciales, pueden provocarse *blobs* o malinterpretaciones y falsos positivos. En estos casos fallan hasta algoritmos como *Haar Cascade* al igual que en situaciones de posiciones que no sean frontales, dónde nuestros métodos sí que funcionan mejor.

Consideramos que el trabajo invertido en desarrollar estas herramientas ha sido lo suficientemente complejo como para darnos cuenta de los puntos fuertes y las limitaciones de los fundamentos de análisis de imagen. Por ello, aunque hoy en día el *Deep Learning*, las redes convolucionales, y otros algoritmos más complejos y con más facilidades puedan funcionar mejor, el punto de vista que te ofrece abordar los problemas desde esta perspectiva sigue pudiendo ser tremendamente útil, tanto para futuros desarrollos, como para entender realmente cómo funciona el nuevo paradigma.

9. Bibliografía.

1. **Pinterest** - Pinterest - <https://es.pinterest.com>
2. **Moodle UPM** - FAI (Ángel Arroyo Castillo) -
<https://moodle.upm.es/titulaciones/oficiales/course/view.php?id=21301>